

CHEATSHEET LENGKAP — LARAVEL CRUD DARI 0 SAMPAI JADI & RUN

Aku pecah jadi 10 tahap, tiap tahap ada **contoh perintah, kode, dan alasan kenapa dipakai.**

STEP 0 — INSTALL PROJECT

Perintah

```
laravel new project-name
```

atau:

```
composer create-project laravel/laravel project-name
```

Kenapa?

Karena ini membuat struktur Laravel lengkap:

- /app = logic
- /routes = URL
- /resources/views = tampilan
- /database/migrations = blueprint database

STEP 1 — SETUP DATABASE

Edit file `.env`:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3307 // kalau pakai port xampp
DB_DATABASE=belajar_crud
DB_USERNAME=root
DB_PASSWORD=root
```

Kenapa harus di `.env`?

Karena:

- Laravel memisahkan config dari code
 - Kode tetap bersih
 - Enkripsi & deploy lebih aman
-

STEP 2 — BUAT MIGRATION (TABEL)

```
php artisan make:migration create_students_table
```

Migration contoh:

```
public function up()
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('nim');
        $table->string('name');
        $table->string('kelas');
        $table->string('gender');
        $table->timestamps();
    });
}
```

Jalankan:

```
php artisan migrate
```

Kenapa pakai migration?

Supaya struktur database:

- Bisa dicatat versi setiap perubahan
 - Bisa ditransfer ke komputer lain
 - Bisa rollback / fresh
-

STEP 3 — BUAT MODEL (Representasi Database di Laravel)

```
php artisan make:model Student
```

Isi Model:

```
class Student extends Model
{
```

```
} protected $fillable = ['nim', 'name', 'kelas', 'gender'];
```

Kenapa harus isi \$fillable?

Agar Laravel mengizinkan insert/update kolom tersebut (mass assignment protection).

Kalau tidak → data tidak masuk.

STEP 4 — BUAT CONTROLLER

```
php artisan make:controller StudentController --resource
```

Kenapa pakai --resource?

Karena Laravel otomatis membuat **7 fungsi CRUD standar**:

Method	Tujuan
index()	tampilkan semua data
create()	form tambah
store()	simpan data baru
show()	detail (opsional)
edit()	form edit
update()	update data
destroy()	delete

STEP 5 — BUAT ROUTE

Di routes/web.php:

```
Route::resource('students', StudentController::class);
```

Kenapa pakai Route::resource?

Karena 1 baris = 7 route CRUD otomatis
tanpa harus ngetik get/post/put/delete manual.

■ STEP 6 — BUAT VIEW INDEX (tampil semua data)

File: resources/views/students/index.blade.php

```
<h1>Data Mahasiswa</h1>

<a href="{{ route('students.create') }}">Tambah Data</a>

<table border="1">
    <tr>
        <th>NIM</th><th>Nama</th><th>Kelas</th><th>Gender</th><th>Aksi</th>
    </tr>

    @foreach($students as $student)
    <tr>
        <td>{{ $student->nim }}</td>
        <td>{{ $student->name }}</td>
        <td>{{ $student->kelas }}</td>
        <td>{{ $student->gender }}</td>
        <td>
            <a href="{{ route('students.edit', $student->id) }}">Edit</a>

            <form action="{{ route('students.destroy', $student->id) }}"
method="POST">
                @csrf
                @method('DELETE')
                <button>Hapus</button>
            </form>
        </td>
    </tr>
    @endforeach
</table>
```

■ STEP 7 — BUAT VIEW CREATE

create.blade.php

```
<form action="{{ route('students.store') }}" method="POST">
    @csrf
    <input name="nim">
    <input name="name">
    <input name="kelas">
    <select name="gender">
        <option value="L">Laki-laki</option>
        <option value="P">Perempuan</option>
    </select>
    <button>Simpan</button>
</form>
```

🔍 Kenapa pakai @csrf?

Untuk keamanan form POST → mencegah hacking CSRF attack.

STEP 8 — LOGIC DI CONTROLLER

store()

```
public function store(Request $request)
{
    $request->validate([
        'nim' => 'required',
        'name' => 'required',
        'kelas' => 'required',
        'gender' => 'required'
    ]);

    Student::create($request->all());

    return redirect()->route('students.index');
}
```

 **Kenapa \$request->validate() ?**

Laravel otomatis:

- Cek input user
- Redirect balik kalau salah
- Kirim error message

 **Kenapa \$request->all() ?**

Untuk memasukkan semua field yang diizinkan (\$fillable).

STEP 9 — VIEW EDIT

edit.blade.php

```
<form action="{{ route('students.update', $student->id) }}" method="POST">
    @csrf
    @method('PUT')

    <input name="nim" value="{{ $student->nim }}">
    <input name="name" value="{{ $student->name }}">
    <input name="kelas" value="{{ $student->kelas }}">
    <select name="gender">
        <option value="L" {{ $student->gender=='L'? 'selected': '' }}>L</option>
```

```
        <option value="P" {{ $student->gender=='P'? 'selected':'' }}>P</option>
    </select>

    <button>Update</button>
</form>
```

Kenapa pakai @method('PUT')?

Karena HTML form tidak mendukung PUT/PATCH/DELETE.

Laravel memalsukan (method spoofing).

STEP 10 — UPDATE LOGIC

```
public function update(Request $request, $id)
{
    $request->validate([
        'nim' => 'required',
        'name' => 'required',
        'kelas' => 'required',
        'gender' => 'required'
    ]);

    $student = Student::findOrFail($id);
    $student->update($request->all());

    return redirect()->route('students.index');
}
```

PENUTUP — CARA RUN

php artisan serve

Buka browser:

 <http://127.0.0.1:8000/students>

BONUS: PENJELASAN ALUR CRUD (BIAR NGERTI SAMPAI A-K-I-R)

Ini alurnya:

1. **User buka URL**
 - Route membaca
 - Kirim ke controller
2. **Controller ambil data dari DB**
 - via Model
3. **Model berhubungan langsung ke database**
 - pakai table sesuai migration
4. **Controller kirim data ke View (Blade)**
 - dengan compact()
5. **View tampilkan HTML + data**
 - user dapat konten
6. **User submit form POST/PUT/DELETE**
 - Controller menerima request → validasi → simpan ke DB → redirect

INI ADALAH MVC versi Laravel.